بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Secure browsing with SSL/TLS

Insecure websites: no confidentiality or authentication hence not suitable for sharing sensitive (e.g banking) information and vulnerable to masquerading

SSL: Secure Sockets Layer, developed by Netscape (1994) for secure browsing, now replaced by IETF standard, Transport Layer Security TLS

TLS: Suite of multiple protocols between application and transport layer not specific for HTTP (e.g. used by openVPN) but mostly used by HTTP (HTTPS)

Secure Sockets Layer (SSL) and Transport Layer Security (TLS)

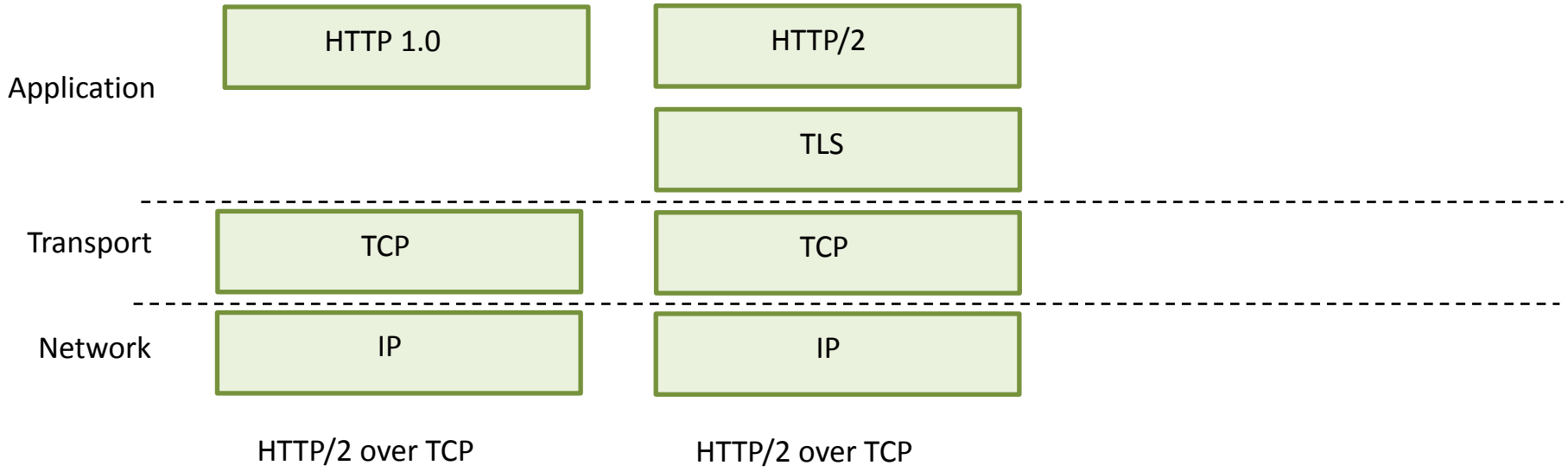
- One of the most widely used security services
- General-purpose service implemented as a set of protocols that rely on TCP
- Subsequently became Internet standard RFC4346: Transport Layer Security (TLS)

Two implementation choices:

- Provided as part of the underlying protocol suite e.g. HTTPS
- Embedded in specific packages **e.g. OpenVPN**

Transport-layer security (TLS)

- TLS provides an API that *any* application can use
- an HTTP view of TLS:



Transport-layer security (TLS)

- widely deployed security protocol above the transport layer
  - supported by almost all browsers, web servers: https (port 443)
- provides:
  - confidentiality: via *symmetric encryption*
  - integrity: via *cryptographic hashing*
  - authentication: via *public key cryptography*

all techniques we have studied!

- history:
  - early research, implementation: secure network programming, secure sockets
  - secure socket layer (SSL) deprecated [2015]
  - TLS 1.3: RFC 8846 [2018]

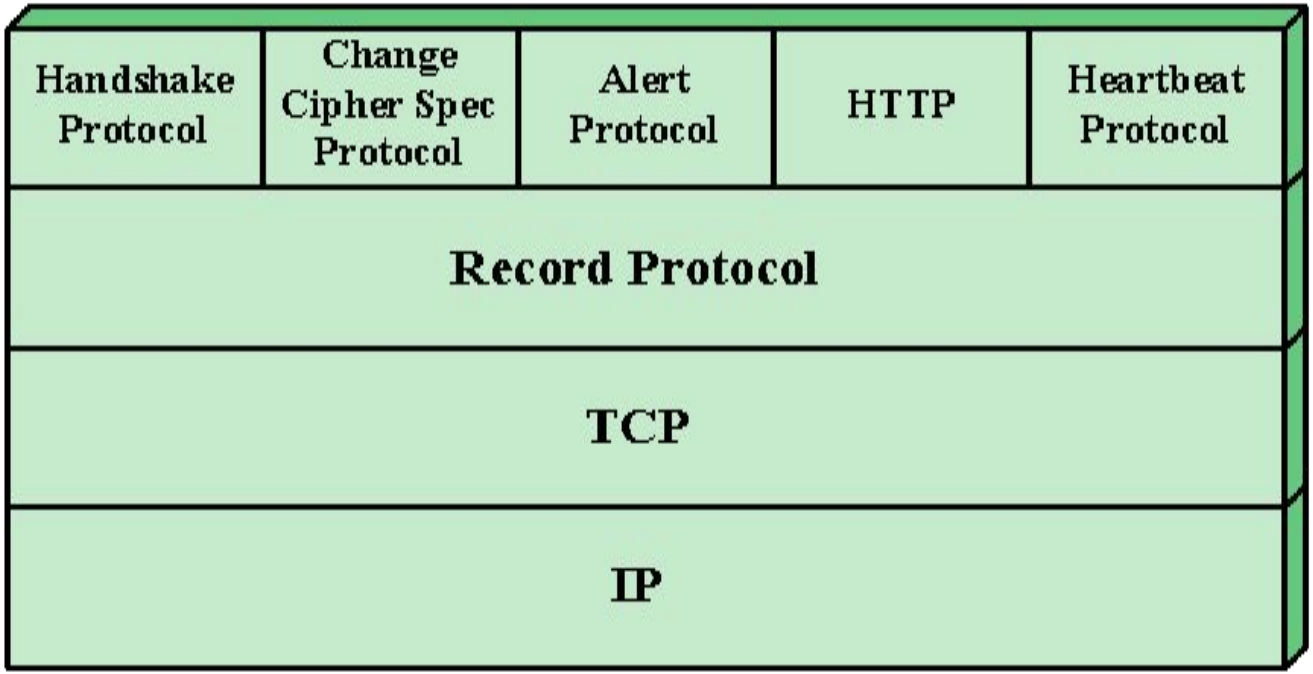


Figure 22.4 SSL/TLS Protocol Stack

Transport-layer security: stages

- handshake: Alice, Bob use their certificates, private keys to authenticate each other, exchange or create shared secret
- key derivation: Alice, Bob use shared secret to derive set of keys
- data transfer: stream data transfer with **encryption**, **authentication** and **optionally compression**
- connection closure: special messages to securely close connection

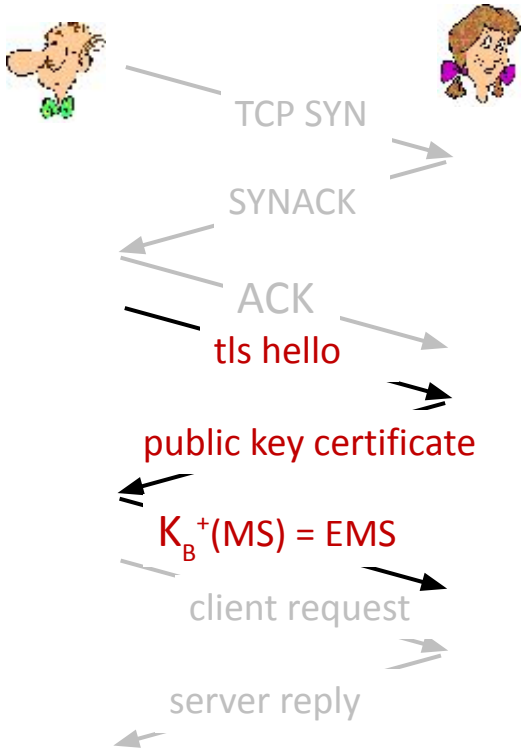
TLS handshake

1. Authenticates the server,
2. Negotiates cipher suite (encryption, authentication and key exchange algorithms)
3. Shares a master session key

Main Steps in TLS handshake:

1. Client(e.g. browser) sends its TLS version and cipher suite preferences
2. Server sends its preferences and its public key certificate
3. Client verifies the certificate using its list of trusted CAs
4. Client generates a random symmetric key as master key for the session and shares it with the server using the server's public key
5. Both notify each other about handshake completion and subsequent encryption.

TLS: initial handshake



TLS handshake phase:

- Bob establishes TCP connection with Alice
- Bob verifies that Alice is really Alice
- Bob sends Alice a master secret key (MS), used to generate all other keys for TLS session

Handshake Protocol



- Most complex part of TLS
- Is used before any application data are transmitted
- Allows server and client to:



- Comprises a series of messages exchanged by client and server
- Exchange has four phases

Handshake Protocol

allows server & client to:

- authenticate each other
- to negotiate encryption & MAC algorithms
- to negotiate cryptographic keys to be used

comprises a series of messages in phases

1. Establish Security Capabilities
2. Server Authentication and Key Exchange
3. Client Authentication and Key Exchange
4. Finish

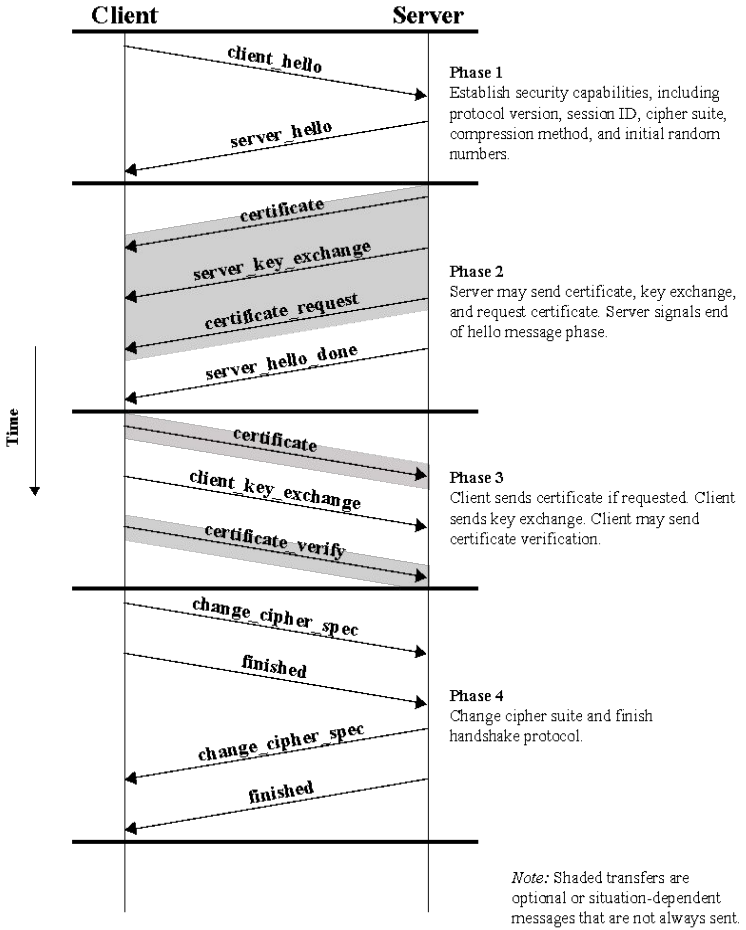


Figure 22.6 Handshake Protocol Action

Table 17.1 *SSL cipher suite list*

Cipher suite	*Key Exchange*	*Encryption*	*Hash*
SSL_NULL_WITH_NULL_NULL	NULL	NULL	NULL
SSL_RSA_WITH_NULL_MD5	RSA	NULL	MD5
SSL_RSA_WITH_NULL_SHA	RSA	NULL	SHA-1
SSL_RSA_WITH_RC4_128_MD5	RSA	RC4	MD5
SSL_RSA_WITH_RC4_128_SHA	RSA	RC4	SHA-1
SSL_RSA_WITH_IDEA_CBC_SHA	RSA	IDEA	SHA-1
SSL_RSA_WITH_DES_CBC_SHA	RSA	DES	SHA-1
SSL_RSA_WITH_3DES_EDE_CBC_SHA	RSA	3DES	SHA-1
SSL_DH_anon_WITH_RC4_128_MD5	DH_anon	RC4	MD5
SSL_DH_anon_WITH_DES_CBC_SHA	DH_anon	DES	SHA-1
SSL_DH_anon_WITH_3DES_EDE_CBC_SHA	DH_anon	3DES	SHA-1
SSL_DHE_RSA_WITH_DES_CBC_SHA	DHE_RSA	DES	SHA-1
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA	DHE_RSA	3DES	SHA-1
SSL_DHE_DSS_WITH_DES_CBC_SHA	DHE_DSS	DES	SHA-1
SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA	DHE_DSS	3DES	SHA-1
SSL_DH_RSA_WITH_DES_CBC_SHA	DH_RSA	DES	SHA-1
SSL_DH_RSA_WITH_3DES_EDE_CBC_SHA	DH_RSA	3DES	SHA-1
SSL_DH_DSS_WITH_DES_CBC_SHA	DH_DSS	DES	SHA-1
SSL_DH_DSS_WITH_3DES_EDE_CBC_SHA	DH_DSS	3DES	SHA-1
SSL_FORTEZZA_DMS_WITH_NULL_SHA	Fortezza	NULL	SHA-1
SSL_FORTEZZA_DMS_WITH_FORTEZZA_CBC_SHA	Fortezza	Fortezza	SHA-1
SSL_FORTEZZA_DMS_WITH_RC4_128_SHA	Fortezza	RC4	SHA-1

TLS: 1.3 cipher suite

- "cipher suite": algorithms that can be used for key generation, encryption, MAC, digital signature
- TLS: 1.3 (2018): more limited cipher suite choice than TLS 1.2 (2008)
  - only 5 choices, rather than 37 choices
  - *requires* Diffie-Hellman (DH) for key exchange, rather than DH or RSA
  - combined encryption and authentication algorithm ("authenticated encryption") for data rather than serial encryption, authentication
  - HMAC uses SHA (256 or 284) cryptographic hash function

TLS: cryptographic keys

- considered bad to use same key for more than one cryptographic function
  - different keys for message authentication code (MAC) and encryption
- four keys:
  - K_c : encryption key for data sent from client to server
  - M_c : MAC key for data sent from client to server
  - K_s : encryption key for data sent from server to client
  - M_s : MAC key for data sent from server to client
- keys derived from key derivation function (KDF)
  - takes master secret and (possibly) some additional random data to create new keys

Change Cipher Spec Protocol

Mainly to indicate start of encrypted data

- One of four TLS specific protocols that use the TLS Record Protocol
- Is the simplest
- **Consists of a single message which consists of a single byte with the value 1 (Indicating that subsequent data will be encrypted)**
- Sole purpose of this message is to cause pending state to be copied into the current state updating the cipher suite in use

Record Protocol Services

message integrity

- using a MAC with shared secret key
- similar to HMAC but with different padding

confidentiality

- using symmetric encryption with a shared secret key defined by Handshake Protocol
- AES, IDEA, RC2-40, DES-40, DES, 3DES, Fortezza, RC4-40, RC4-128
- message (optionally) compressed before encryption

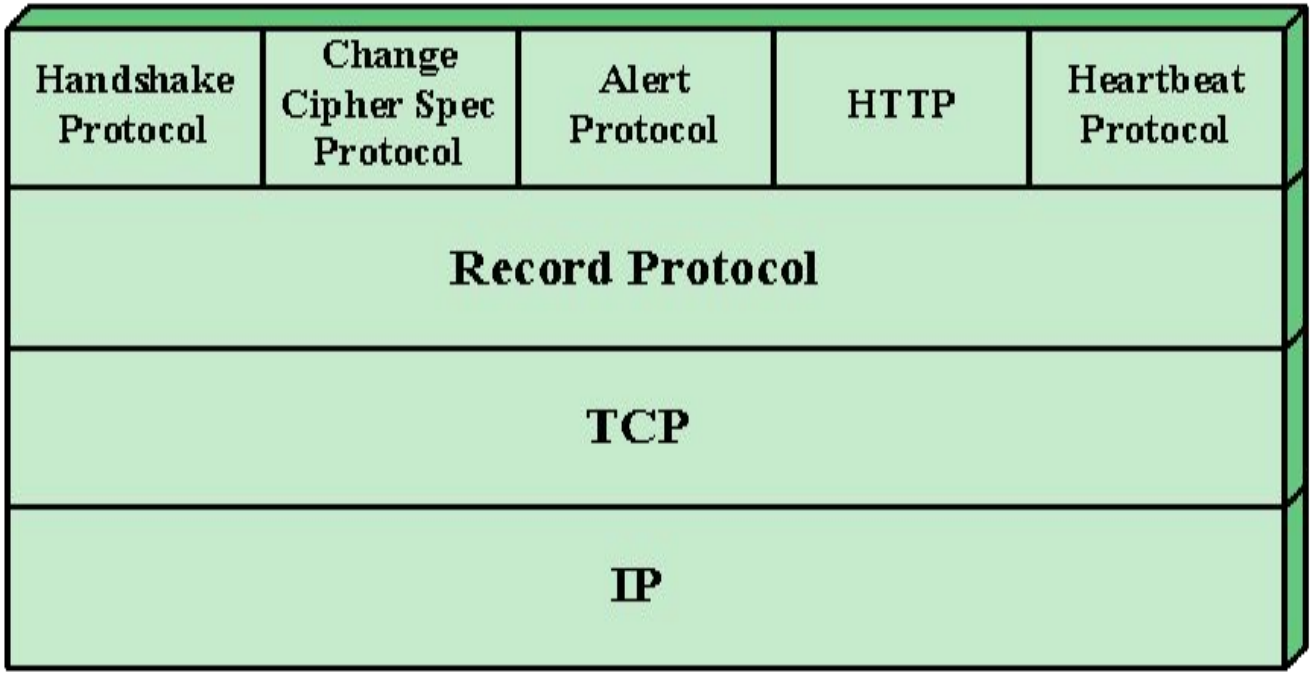
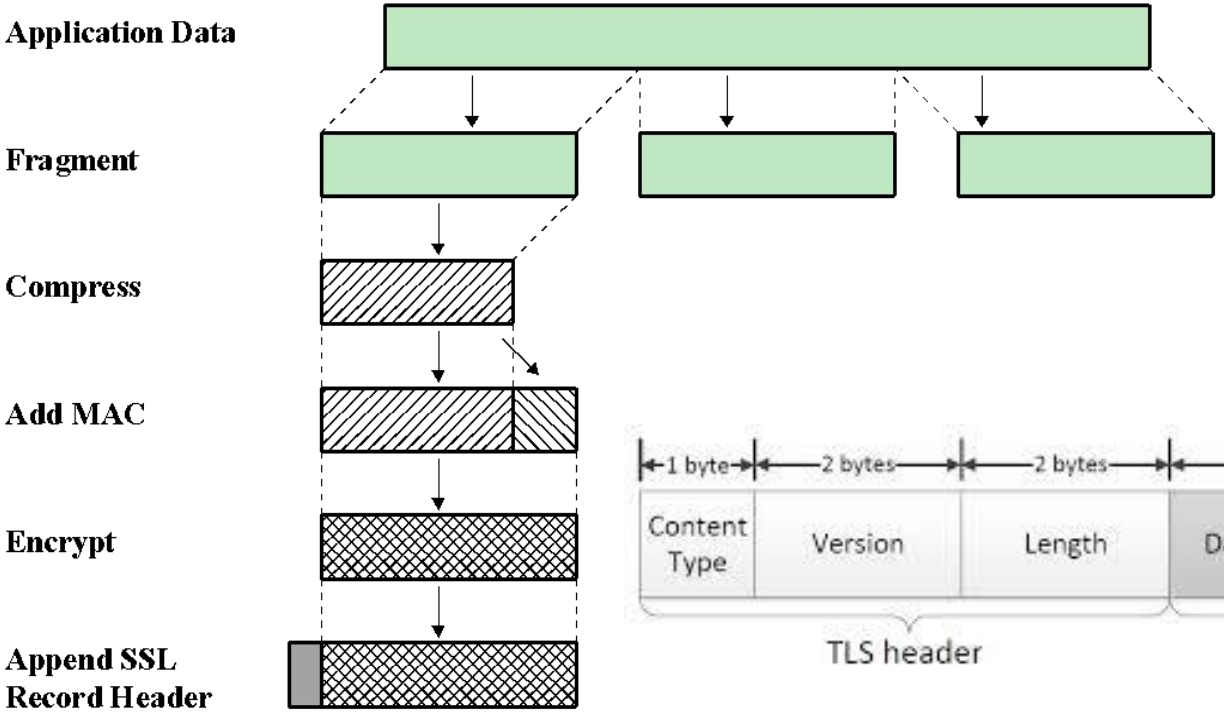
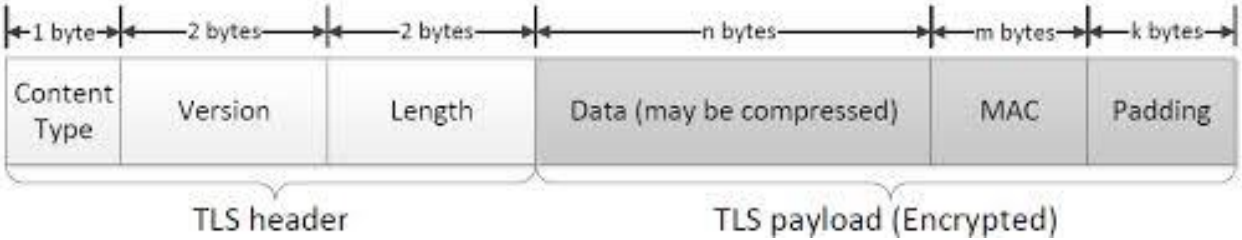


Figure 22.4 SSL/TLS Protocol Stack





Record (TLS) Header

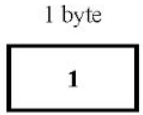
Figure 22.5 TLS Record Protocol Operation

Packet Format
Record Header

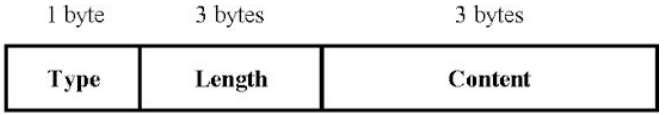
NA	TLS 1.3
0x303	TLS 1.2
0x302	TLS 1.1
0x301	TLS 1.0
0x300	SSL V3.0

Byte [0] Content Type	Byte [1:2] Version	Byte [3:4] Length	Byte [5: n] Payload

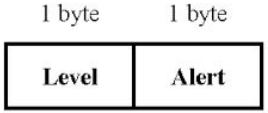
0x14	ChangeCipherSpec
0x15	SSL Alert
0x16	Handshake
0x17	ApplicationData



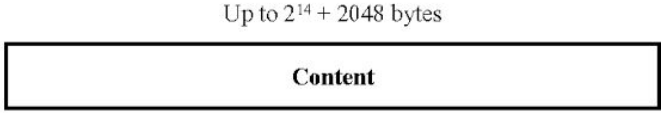
(a) Change Cipher Spec Protocol



(c) Handshake Protocol



(b) Alert Protocol



(d) Other Upper Layer Protocol as TLS Payload

TLS: encrypting data

Traditionally TCP segments application data stream as per it's window but for MAC, TLS segments data into records before handing over to TCP

- recall: TCP provides data *byte stream* abstraction
- Q: can we encrypt data in-stream as written into TCP socket?
  - *A:* where would MAC go? If at end, no message integrity until all data received and connection closed!
  - *solution:* break stream in series of "records"
    - **each client-to-server record carries a MAC, created using M_c**
    - receiver can act on each record as it arrives
    - tls record encrypted using symmetric key, $K_{c,}$ passed to TCP:



TLS: encrypting data (re order replay)

- possible attacks on data stream?
  - *re-ordering:* man-in middle intercepts TCP segments and reorders (manipulating sequence #s in unencrypted TCP header)
- solutions:
  - **use TLS sequence numbers (data, TLS-seq-# incorporated into MAC)**
  - **MAC now computed using data, type, sequence #**

e.g : MAC = HMAC(key, seq_num || header || data)

K_c(

)

Sequence number not sent with message but just used to compute MAC and maintained by both sides incrementing on sent /received messages. if sequence number does not correspond to the current message, then message authentication will fail, and the receiver will demand the sender to re-send the message.

TLS: encrypting data (replay)

possible attacks on data stream?

- *replay against authentication*

solutions:

use nonce (new random number) to overcome replay attack

Shared during handshake and used for key derivation function making the keys usable only for the session

Nonce can also be used as initialization vector for block chaining mode e.g. CBC

TCP, TLS Connections and TLS Session

TLS runs **over TCP** establishing the underlying **TCP Connection**

Immediately following the TCP Handshake, the **TLS Handshake** is initiated negotiated cryptographic parameters (the cipher suite and master secret) to **create an encrypted** secure channel that is the **TLS Connection**.

TLS Connection is short-lived and lasts only as long as the underlying TCP Connection remains open.

The parameters negotiated during the TLS Handshake are stored as **TLS Session** data typically identified by a **Session ID**

The stored TLS Session is **long-lived** (e.g. **expiry** configured by the server between 10 minutes and 10 hours)

Subsequent new **TLS Connections** between the same client and server can **reuse** this stored Session data.

When the client reconnects, it sends the stored **Session ID** in its initial **ClientHello** message.

If the server finds the corresponding active session state, It **skips the costly public-key cryptography** phase of the full handshake.

It immediately uses the stored **Master Secret** to derive the new connection keys, quickly establishing a **new, secure TLS Connection** using the parameters from the old **TLS Session**.

TLS Concepts

TLS Session

- An association between a client and a server
- Define a set of cryptographic security parameters
- **Can be shared among multiple connections**
- Used to avoid the expensive negotiation of new security parameters for each connection

TLS Connection

- A transport (in the OSI layering model definition) that provides a suitable type of service
- Peer-to-peer relationships
- Transient
- **Every connection is associated with one session**

HTTP user authentication

Besides the end point authentication via certificates within the initial handshake and Message authentication for the entire TLS session by HMAC

Over a TLS session:

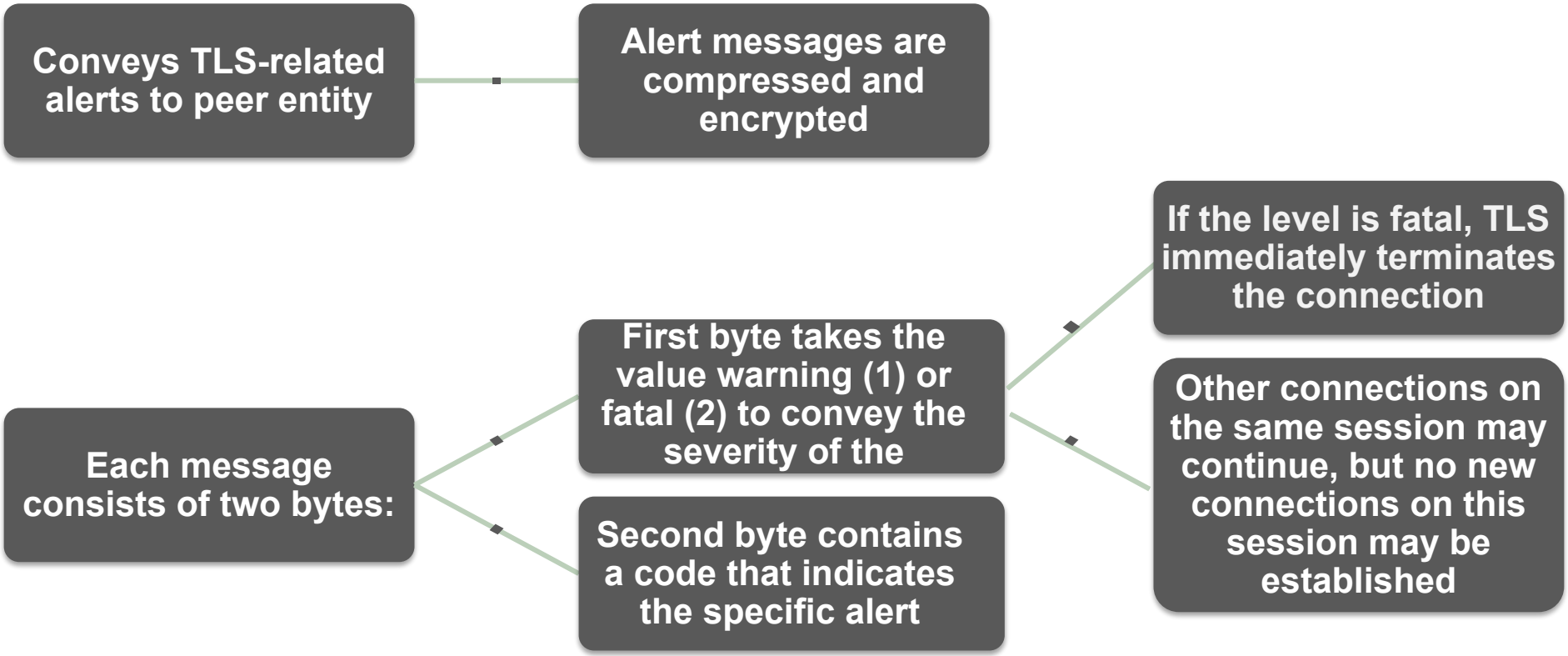
For **HTTP user authentication (login)**, Username/password or 2-factor code is sent inside an **HTTP POST request**, protected by TLS encryption.

The user credentials are usually stored on the server as Hashed (with salt)

Upon authentication Server generates a **session ID/token** to represent the authenticated user

The HTTP session is maintained via HTTP cookies
All HTTP messages traveling inside the same encrypted TLS tunnel.

Alert Protocol



Alert Protocol

conveys TLS-related alerts to peer entity

severity

- warning or fatal

specific alert

- fatal: unexpected message, bad record mac, decompression failure, handshake failure, illegal parameter
- warning: **close_notify**, no certificate, bad certificate, unsupported certificate, certificate revoked, certificate expired, certificate unknown

compressed & encrypted like all TLS data

TLS: connection close

- truncation attack:
  - attacker forges TCP connection close segment
  - one or both sides thinks there is less data than there actually is
- solution: TLS Connection closure alert (close_notify) included as an for the ALERT protocol messages, with one type for closure

TLS uses the close_notify alert (an ALERT-protocol message protected by MAC) to securely close a TLS connection. This alert must be received *before* the TCP FIN.

If the TCP connection closes without a close_notify alert, TLS treats this as an error and reports an abnormal termination to the application.

Heartbeat Protocol

- A periodic signal generated by hardware or software to indicate normal operation or to synchronize other parts of a system
- Typically used to monitor the availability of a protocol entity
- Defined in 2012 in RFC 6250
- Runs on top of the TLS Record Protocol
- **Use is established during Phase 1 of the Handshake Protocol**
- **Each peer indicates whether it supports heartbeats**
- Serves two purposes:
  - Assures the sender that the recipient is still alive
  - Generates activity across the connection during idle periods

Idle period can cause to close TCP connections, Firewall sessions, NAT entries, that Heartbeat prevents

HTTPS
(HTTP over SSL)

- Combination of HTTP and SSL to implement secure communication between a Web browser and a Web server
- Built into all modern Web browsers
  - Search engines do not support HTTPS
  - URL addresses begin with https://
- Documented in RFC 2818, HTTP Over TLS
- Agent acting as the HTTP client also acts as the TLS client
- Closure of an HTTPS connection requires that TLS close the connection with the peer TLS entity on the remote side, which will involve closing the underlying TCP connection

OpenSSL

Openssl: **open-source cryptographic library and TLS toolkit (written in C)**

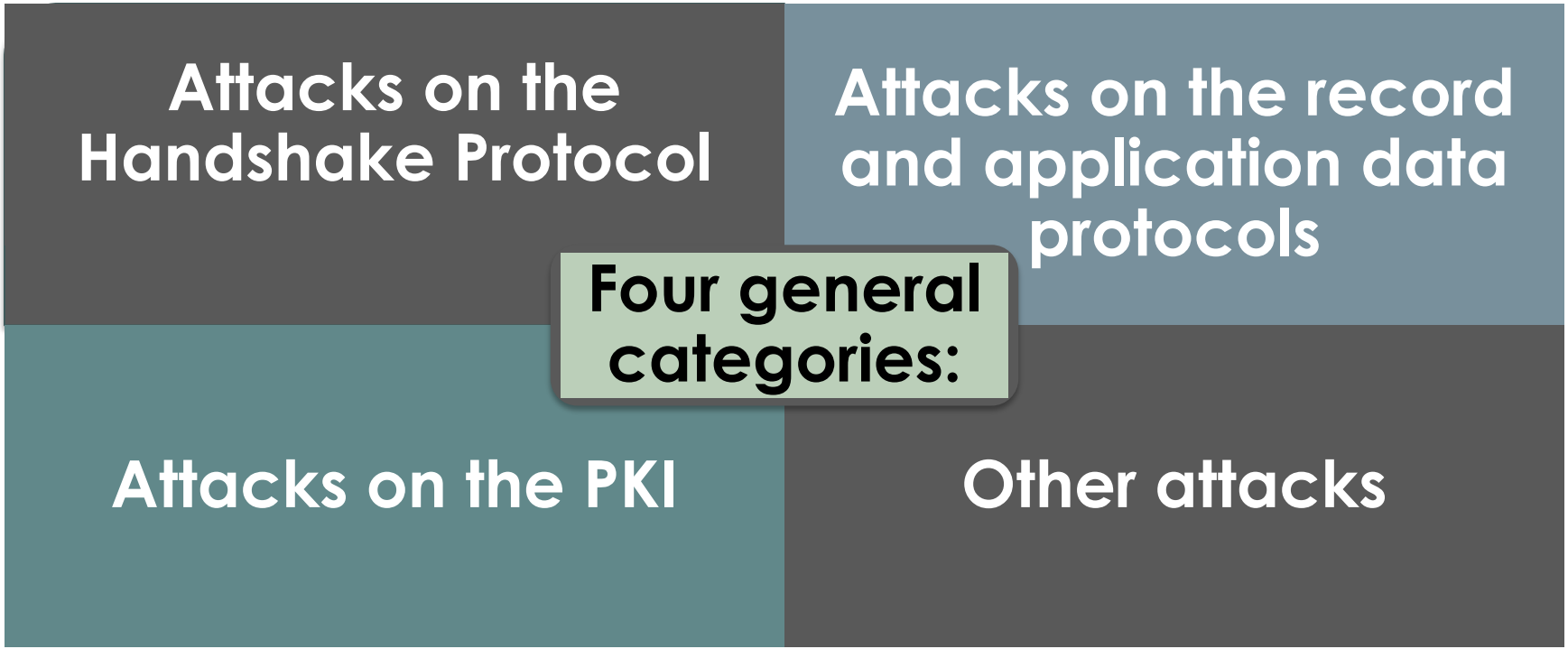
Implements TLS protocols, certificates, encryption, decryption, and cryptographic functions.

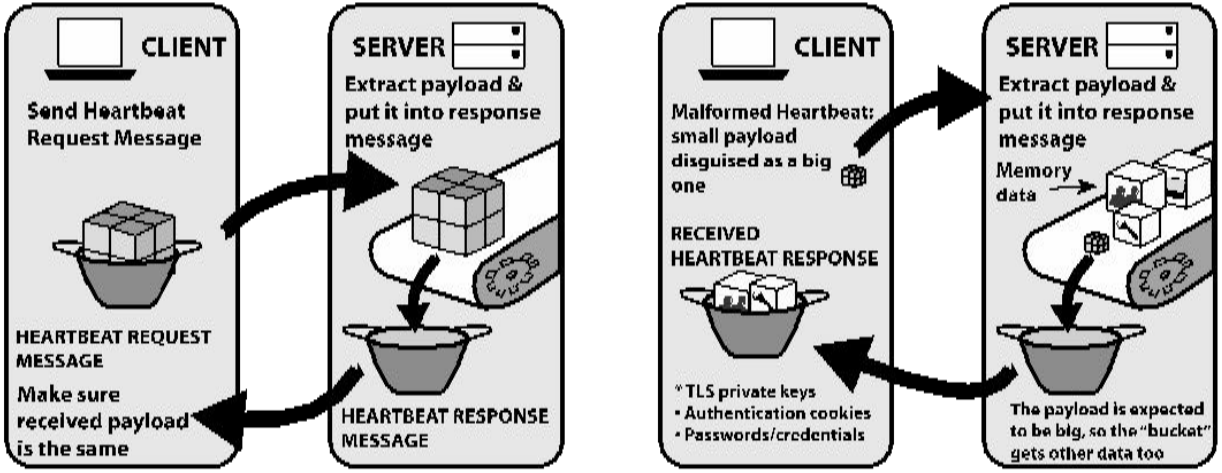
Usage Examples

Apache Web Server: to implement SSL/TLS for HTTPS

OpenVPN: to provide VPN connections using TLS

SSL/TLS Attacks





(a) How TLS Heartbeat Protocol works

(b) How TLS Heartbleed exploit works

Figure 22.7 The Heartbleed Exploit
Source: BAE Systems

Heartbeat_request message includes payload length, payload and padding fields.

Not a design flaw in the TLS rather a bug in the open-source OpenSSL implementation of the Heartbeat Protocol: Before the bug was fixed, the OpenSSL version of the Heartbeat Protocol worked as follows: **The software reads the incoming request message and allocates a buffer large enough to hold the message header, the payload**. It then **overwrites the current contents of the buffer with the incoming message**, changes the first byte to indicate the response message type, then **transmits a response message, which includes the payload length field and the same payload**.

the software **does not check the message length of the incoming message**. As a result, **an adversary can send a message that indicates the maximum payload length (64 KB) but only includes the minimum payload (16 bytes).** This means that all of the **buffer is not overwritten and whatever was in memory will be sent to the requestor.**